Module 32

Application Design and Development/2: Web Applications

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
ppd@cse.iitkgp.ac.in

March 17, 2026

Module Recap

- Had a glimpse of Application Programs across various sectors

Module Recap

- Had a glimpse of Application Programs across various sectors
- Understood the typical architecture for an application

Module Recap

- Had a glimpse of Application Programs across various sectors
- Understood the typical architecture for an application
- Studies the classification and evolution of the architectures

Module Recap

- Had a glimpse of Application Programs across various sectors
- Understood the typical architecture for an application
- Studies the classification and evolution of the architectures
- Glimpsed at architecture for a few sample applications

Module Objectives

Goals

- To familiarize with the fundamentals notions and technologies of Web

Module Objectives

Goals

- To familiarize with the fundamentals notions and technologies of Web
- To learn about scripting

Module Objectives

Goals

- To familiarize with the fundamentals notions and technologies of Web
- To learn about scripting
- To learn about Servlets

Module Outline

- Web Fundamentals and Scripting

Module Outline

- Web Fundamentals and Scripting
- Servlets

Web Fundamentals

The World Wide Web

- The Web is a distributed information system based on hypertext

The World Wide Web

- The Web is a distributed information system based on hypertext
- Most Web documents are hypertext documents formatted via the HyperText Markup Language (HTML)

The World Wide Web

- The Web is a distributed information system based on hypertext
- Most Web documents are hypertext documents formatted via the HyperText Markup Language (HTML)
- HTML documents contain

The World Wide Web

- The Web is a distributed information system based on hypertext
- Most Web documents are hypertext documents formatted via the HyperText Markup Language (HTML)
- HTML documents contain
  - text along with font specifications, and other formatting instructions

The World Wide Web

- The Web is a distributed information system based on hypertext
- Most Web documents are hypertext documents formatted via the HyperText Markup Language (HTML)
- HTML documents contain
  - text along with font specifications, and other formatting instructions
  - hypertext links to other documents, which can be associated with regions of the text

The World Wide Web

- The Web is a distributed information system based on hypertext
- Most Web documents are hypertext documents formatted via the HyperText Markup Language (HTML)
- HTML documents contain
  - text along with font specifications, and other formatting instructions
  - hypertext links to other documents, which can be associated with regions of the text
  - forms, enabling users to enter data which can then be sent back to the Web server

Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource Locators (URLs).

Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource Locators (URLs).
- **URL example:** `http://www.acm.org/sigmod`

Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource Locators (URLs).
- **URL example:** `http://www.acm.org/sigmod`
  - The first part indicates how the document is to be accessed (protocol): 'http' indicates HTTP

Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource Locators (URLs).
- **URL example:** `http://www.acm.org/sigmod`
  - The first part indicates how the document is to be accessed (protocol): 'http' indicates HTTP
  - The second part gives the unique name of a machine on the Internet

Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource Locators (URLs).
- **URL example:** `http://www.acm.org/sigmod`
  - The first part indicates how the document is to be accessed (protocol): 'http' indicates HTTP
  - The second part gives the unique name of a machine on the Internet
  - The rest of the URL identifies the document within the machine

Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource Locators (URLs).
- **URL example:** `http://www.acm.org/sigmod`
  - The first part indicates how the document is to be accessed (protocol): 'http' indicates HTTP
  - The second part gives the unique name of a machine on the Internet
  - The rest of the URL identifies the document within the machine
- The local identification can be:

Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource Locators (URLs).
- **URL example:** `http://www.acm.org/sigmod`
  - The first part indicates how the document is to be accessed (protocol): 'http' indicates HTTP
  - The second part gives the unique name of a machine on the Internet
  - The rest of the URL identifies the document within the machine
- The local identification can be:
  - The path name of a file on the machine

Uniform Resources Locators

- On the Web, functionality of pointers is provided by Uniform Resource Locators (URLs).
- **URL example:** `http://www.acm.org/sigmod`
  - The first part indicates how the document is to be accessed (protocol): 'http' indicates HTTP
  - The second part gives the unique name of a machine on the Internet
  - The rest of the URL identifies the document within the machine
- The local identification can be:
  - The path name of a file on the machine
  - An identifier of a program plus arguments to be passed to the program

URI, URL, and URN

Definitions

- **Uniform Resource Identifier (URI)**: The overarching concept

URI, URL, and URN

Definitions

- **Uniform Resource Identifier (URI)**: The overarching concept
- **Uniform Resource Locator (URL)**: Locates the resource

URI, URL, and URN

Definitions

- **Uniform Resource Identifier (URI)**: The overarching concept
- **Uniform Resource Locator (URL)**: Locates the resource
- **Uniform Resource Name (URN)**: Names the resource

URI, URL, and URN

Definitions

- **Uniform Resource Identifier (URI)**: The overarching concept
- **Uniform Resource Locator (URL)**: Locates the resource
- **Uniform Resource Name (URN)**: Names the resource

- URIs can be classified as locators (URLs), or as names (URNs), or as both

URI, URL, and URN

Definitions

- **Uniform Resource Identifier (URI)**: The overarching concept
- **Uniform Resource Locator (URL)**: Locates the resource
- **Uniform Resource Name (URN)**: Names the resource

- URIs can be classified as locators (URLs), or as names (URNs), or as both
- URN functions like a person's name; URL resembles that person's street address

URI, URL, and URN

Definitions

- **Uniform Resource Identifier (URI)**: The overarching concept
- **Uniform Resource Locator (URL)**: Locates the resource
- **Uniform Resource Name (URN)**: Names the resource

- URIs can be classified as locators (URLs), or as names (URNs), or as both
- URN functions like a person's name; URL resembles that person's street address
- URN defines an item's identity, while the URL provides a method for finding it

HTML and HTTP

- HTML provides formatting, hypertext link, and image display features

HTML and HTTP

- HTML provides formatting, hypertext link, and image display features
  - including tables, stylesheets (to alter default formatting), etc.

HTML and HTTP

- HTML provides formatting, hypertext link, and image display features
  - including tables, stylesheets (to alter default formatting), etc.
- HTML also provides input features

HTML and HTTP

- HTML provides formatting, hypertext link, and image display features
  - including tables, stylesheets (to alter default formatting), etc.
- HTML also provides input features
  - Select from a set of options: Pop-up menus, radio buttons, check lists

HTML and HTTP

- HTML provides formatting, hypertext link, and image display features
  - including tables, stylesheets (to alter default formatting), etc.
- HTML also provides input features
  - Select from a set of options: Pop-up menus, radio buttons, check lists
  - Enter values: Text boxes

HTML and HTTP

- HTML provides formatting, hypertext link, and image display features
  - including tables, stylesheets (to alter default formatting), etc.
- HTML also provides input features
  - Select from a set of options: Pop-up menus, radio buttons, check lists
  - Enter values: Text boxes
- Filled in input sent back to the server, to be acted upon by an executable at the server

HTML and HTTP

- HTML provides formatting, hypertext link, and image display features
  - including tables, stylesheets (to alter default formatting), etc.
- HTML also provides input features
  - Select from a set of options: Pop-up menus, radio buttons, check lists
  - Enter values: Text boxes
- Filled in input sent back to the server, to be acted upon by an executable at the server
- HyperText Transfer Protocol (HTTP) used for communication with the Web server

Sample HTML

```
<html>
<body>
<table border>
<tr><th>ID</th><th>Name</th><th>Department</th></tr>
<tr><td>00128</td><td>Zhang</td><td>Comp. Sci.</td></tr>
...
</table>
<form action='PersonQuery' method='get'>
Search for:
<select name='persontype'>
<option value='student' selected>Student</option>
<option value='instructor'>Instructor</option>
</select>
<br>
Name:
```

HTTP and Sessions

- **The HTTP protocol is connectionless**

HTTP and Sessions

- **The HTTP protocol is connectionless**
  - Once the server replies, it closes the connection and forgets the request

HTTP and Sessions

- **The HTTP protocol is connectionless**
  - Once the server replies, it closes the connection and forgets the request
  - Unix logins and JDBC/ODBC connections stay connected, retaining authentication

HTTP and Sessions

- **The HTTP protocol is connectionless**
  - Once the server replies, it closes the connection and forgets the request
  - Unix logins and JDBC/ODBC connections stay connected, retaining authentication
  - **Motivation:** reduces load on server (OS has tight limits on open connections)

HTTP and Sessions

- **The HTTP protocol is connectionless**
  - Once the server replies, it closes the connection and forgets the request
  - Unix logins and JDBC/ODBC connections stay connected, retaining authentication
  - **Motivation:** reduces load on server (OS has tight limits on open connections)
- Information services need session information

HTTP and Sessions

- **The HTTP protocol is connectionless**
  - Once the server replies, it closes the connection and forgets the request
  - Unix logins and JDBC/ODBC connections stay connected, retaining authentication
  - **Motivation:** reduces load on server (OS has tight limits on open connections)
- Information services need session information
  - For example, user authentication should be done only once per session

HTTP and Sessions

- **The HTTP protocol is connectionless**
  - Once the server replies, it closes the connection and forgets the request
  - Unix logins and JDBC/ODBC connections stay connected, retaining authentication
  - **Motivation:** reduces load on server (OS has tight limits on open connections)
- Information services need session information
  - For example, user authentication should be done only once per session
- **Solution:** use a cookie

Sessions and Cookies

- A **cookie** is a small piece of text containing identifying information

Sessions and Cookies

- A **cookie** is a small piece of text containing identifying information
  - Sent by server to browser on first interaction to identify session

Sessions and Cookies

- A **cookie** is a small piece of text containing identifying information
  - Sent by server to browser on first interaction to identify session
  - Sent by browser back to the server that created the cookie on further interactions (part of HTTP protocol)

Sessions and Cookies

- A **cookie** is a small piece of text containing identifying information
  - Sent by server to browser on first interaction to identify session
  - Sent by browser back to the server that created the cookie on further interactions (part of HTTP protocol)
- Server saves information about cookies it issued, and can use it when serving a request

Sessions and Cookies

- A **cookie** is a small piece of text containing identifying information
  - Sent by server to browser on first interaction to identify session
  - Sent by browser back to the server that created the cookie on further interactions (part of HTTP protocol)
- Server saves information about cookies it issued, and can use it when serving a request
  - For example, authentication information, and user preferences

Sessions and Cookies

- A **cookie** is a small piece of text containing identifying information
  - Sent by server to browser on first interaction to identify session
  - Sent by browser back to the server that created the cookie on further interactions (part of HTTP protocol)
- Server saves information about cookies it issued, and can use it when serving a request
  - For example, authentication information, and user preferences
- Cookies can be stored permanently or for a limited time

Web Browser

- A web browser is application software for accessing the World Wide Web

Web Browser

- A web browser is application software for accessing the World Wide Web
- A web browser fetches content from the Web and displays it on a user's device

Web Browser

- A web browser is application software for accessing the World Wide Web
- A web browser fetches content from the Web and displays it on a user's device
- A browser or rendering engine transforms HTML documents and other resources into an interactive visual representation

Web Browser

- A web browser is application software for accessing the World Wide Web
- A web browser fetches content from the Web and displays it on a user's device
- A browser or rendering engine transforms HTML documents and other resources into an interactive visual representation
  - This includes image and video formats supported by the browser

Web Browser

- A web browser is application software for accessing the World Wide Web
- A web browser fetches content from the Web and displays it on a user's device
- A browser or rendering engine transforms HTML documents and other resources into an interactive visual representation
  - This includes image and video formats supported by the browser
- Web pages usually contain hyperlinks to other pages and resources

Web Browser

- A web browser is application software for accessing the World Wide Web
- A web browser fetches content from the Web and displays it on a user's device
- A browser or rendering engine transforms HTML documents and other resources into an interactive visual representation
  - This includes image and video formats supported by the browser
- Web pages usually contain hyperlinks to other pages and resources
- Web browsers are used on a range of devices, including desktops, laptops, tablets, and smartphones

Web Browser

- A web browser is application software for accessing the World Wide Web
- A web browser fetches content from the Web and displays it on a user's device
- A browser or rendering engine transforms HTML documents and other resources into an interactive visual representation
  - This includes image and video formats supported by the browser
- Web pages usually contain hyperlinks to other pages and resources
- Web browsers are used on a range of devices, including desktops, laptops, tablets, and smartphones
  - In 2020, an estimated 4.9 billion people used a browser

Web Browser

- A web browser is application software for accessing the World Wide Web
- A web browser fetches content from the Web and displays it on a user's device
- A browser or rendering engine transforms HTML documents and other resources into an interactive visual representation
  - This includes image and video formats supported by the browser
- Web pages usually contain hyperlinks to other pages and resources
- Web browsers are used on a range of devices, including desktops, laptops, tablets, and smartphones
  - In 2020, an estimated 4.9 billion people used a browser
  - Most used browser: Google Chrome with 64% global market share; Safari with 19%

Web Servers

- A web server is software and underlying hardware that accepts requests via HTTP or HTTPS

Web Servers

- A web server is software and underlying hardware that accepts requests via HTTP or HTTPS
- A web browser or crawler requests a specific resource using HTTP, and the server responds with the content or an error message

Web Servers

- A web server is software and underlying hardware that accepts requests via HTTP or HTTPS
- A web browser or crawler requests a specific resource using HTTP, and the server responds with the content or an error message
- The document name in a URL may identify an executable program that, when run, generates a HTML document

Web Servers

- A web server is software and underlying hardware that accepts requests via HTTP or HTTPS
- A web browser or crawler requests a specific resource using HTTP, and the server responds with the content or an error message
- The document name in a URL may identify an executable program that, when run, generates a HTML document
- To install a new service on the Web, one simply needs to create and install an executable that provides that service

Web Servers

- A web server is software and underlying hardware that accepts requests via HTTP or HTTPS
- A web browser or crawler requests a specific resource using HTTP, and the server responds with the content or an error message
- The document name in a URL may identify an executable program that, when run, generates a HTML document
- To install a new service on the Web, one simply needs to create and install an executable that provides that service
- Common Gateway Interface (CGI): a standard interface between web and application server

Web Services

- Allow data on Web to be accessed using remote procedure call mechanism

Web Services

- Allow data on Web to be accessed using remote procedure call mechanism
- Two approaches are widely used

Web Services

- Allow data on Web to be accessed using remote procedure call mechanism
- Two approaches are widely used
  - **REST (Representation State Transfer):** allows use of standard HTTP request to a URL to execute a request and return data; encoded in XML or JSON

Web Services

- Allow data on Web to be accessed using remote procedure call mechanism
- Two approaches are widely used
  - **REST (Representation State Transfer):** allows use of standard HTTP request to a URL to execute a request and return data; encoded in XML or JSON
  - **Big Web Services:** uses XML representation for sending request data and returning results; standard protocol layer built on top of HTTP

Scripting

Two-Tier Web Architecture

Three-Tier Web Architecture

Client-Side Scripting and Validation

- Advantages of execution at client

Client-Side Scripting and Validation

- Advantages of execution at client
  - Offloads work from server

Client-Side Scripting and Validation

- Advantages of execution at client
  - Offloads work from server
  - Provides more flexible user interface

Client-Side Scripting and Validation

- Advantages of execution at client
  - Offloads work from server
  - Provides more flexible user interface
- **Disadvantages**

Client-Side Scripting and Validation

- Advantages of execution at client
  - Offloads work from server
  - Provides more flexible user interface
- **Disadvantages**
  - Client functionality is not guaranteed (e.g., Javascript may be disabled)

Client-Side Scripting and Validation

- Advantages of execution at client
  - Offloads work from server
  - Provides more flexible user interface
- **Disadvantages**
  - Client functionality is not guaranteed (e.g., Javascript may be disabled)
  - Browser may not support newer language features

Client-Side Scripting and Validation

- Advantages of execution at client
  - Offloads work from server
  - Provides more flexible user interface
- **Disadvantages**
  - Client functionality is not guaranteed (e.g., Javascript may be disabled)
  - Browser may not support newer language features
  - Same application looks/behaves differently in different browsers

Client-Side Scripting and Validation

- Advantages of execution at client
  - Offloads work from server
  - Provides more flexible user interface
- **Disadvantages**
  - Client functionality is not guaranteed (e.g., Javascript may be disabled)
  - Browser may not support newer language features
  - Same application looks/behaves differently in different browsers
  - Client code can be hacked; security checks must always be verified at the server

Javascript

- **Javascript**
  - Language very similar to Java
  - Typically executed by the browser
- Javascript can
  - validate input,
  - interact with user,
  - alter display based on user interaction
- Document Object Model (DOM)
  - Tree representation of HTML document
  - Nodes of tree can be accessed via Javascript
  - ▷ `document.getElementById('some_id')`
  - And updated
  - ▷ `document.getElementById('some_id').innerHTML = "<b>Hello</b>"`
  - Forms can be checked, and values updated

Javascript (2): Example

```
<html>
<head>
<script type="text/javascript">
function validate() {
  var creds = document.getElementById("CREDIT_CARD").value;
  if (creds.length != 16) {
    alert("Credit card must have 16 digits");
    return false;
  } else {
    return true;
  }
}
</script>
</head>
<body>
```

Server-Side Scripting

- Server-side scripting simplifies the task of connecting a database to the Web

Server-Side Scripting

- Server-side scripting simplifies the task of connecting a database to the Web
  - Define an HTML document with embedded executable code/SQL queries

Server-Side Scripting

- Server-side scripting simplifies the task of connecting a database to the Web
  - Define an HTML document with embedded executable code/SQL queries
  - Input values from HTML forms can be used directly in the embedded code/SQL queries

Server-Side Scripting

- Server-side scripting simplifies the task of connecting a database to the Web
  - Define an HTML document with embedded executable code/SQL queries
  - Input values from HTML forms can be used directly in the embedded code/SQL queries
  - When the document is requested, the Web server executes the embedded code/SQL queries to generate the actual HTML document, which is sent back to the client

Server Side Context

- **A Web Application Layer**

Server Side Context

- **A Web Application Layer**
  - Java Servlets

Server Side Context

- **A Web Application Layer**
  - Java Servlets
  - Java Server Pages (JSP)

Server Side Context

- **A Web Application Layer**
  - Java Servlets
  - Java Server Pages (JSP)
  - Application Programming Interface (API) – Java code with annotations

Server Side Context

- **A Web Application Layer**
  - Java Servlets
  - Java Server Pages (JSP)
  - Application Programming Interface (API) – Java code with annotations
  - Python

Server Side Context

- **A Web Application Layer**
  - Java Servlets
  - Java Server Pages (JSP)
  - Application Programming Interface (API) – Java code with annotations
  - Python
  - Node JS

Server Side Context

- **A Web Application Layer**
  - Java Servlets
  - Java Server Pages (JSP)
  - Application Programming Interface (API) – Java code with annotations
  - Python
  - Node JS
- Web application layer maps HTTP requests to method invocations on a class

Server Side Context

- **A Web Application Layer**
  - Java Servlets
  - Java Server Pages (JSP)
  - Application Programming Interface (API) – Java code with annotations
  - Python
  - Node JS
- Web application layer maps HTTP requests to method invocations on a class
  - Maps parameters from the HTTP request to arguments of the method

Server Side Context

- **A Web Application Layer**
  - Java Servlets
  - Java Server Pages (JSP)
  - Application Programming Interface (API) – Java code with annotations
  - Python
  - Node JS
- Web application layer maps HTTP requests to method invocations on a class
  - Maps parameters from the HTTP request to arguments of the method
  - Expects method to return a string or an object mapped to a string (such as HTML or JSON)

Servlets

Servlets

- Java classes that run in a Web server

Servlets

- Java classes that run in a Web server
- Provide an API for HTTP interaction

Servlets

- Java classes that run in a Web server
- Provide an API for HTTP interaction
  - Can process HTTP/HTTPS requests

Servlets

- Java classes that run in a Web server
- Provide an API for HTTP interaction
  - Can process HTTP/HTTPS requests
  - Obtain parameters

Servlets

- Java classes that run in a Web server
- Provide an API for HTTP interaction
  - Can process HTTP/HTTPS requests
  - Obtain parameters
  - Return results to the HTTP client (usually a web browser)

Servlets

- Java classes that run in a Web server
- Provide an API for HTTP interaction
  - Can process HTTP/HTTPS requests
  - Obtain parameters
  - Return results to the HTTP client (usually a web browser)
  - Maintain information across requests (Sessions)

Servlets

- Java classes that run in a Web server
- Provide an API for HTTP interaction
  - Can process HTTP/HTTPS requests
  - Obtain parameters
  - Return results to the HTTP client (usually a web browser)
  - Maintain information across requests (Sessions)
- **Example:** `javax.servlet.http.HttpServlet`

Servlets

- Java classes that run in a Web server
- Provide an API for HTTP interaction
  - Can process HTTP/HTTPS requests
  - Obtain parameters
  - Return results to the HTTP client (usually a web browser)
  - Maintain information across requests (Sessions)
- **Example:** `javax.servlet.http.HttpServlet`
  - `doGet(request, response)`

Servlets

- Java classes that run in a Web server
- Provide an API for HTTP interaction
  - Can process HTTP/HTTPS requests
  - Obtain parameters
  - Return results to the HTTP client (usually a web browser)
  - Maintain information across requests (Sessions)
- **Example:** `javax.servlet.http.HttpServlet`
  - `doGet(request, response)`
  - `doPost(request, response)`

Servlets

- Java classes that run in a Web server
- Provide an API for HTTP interaction
  - Can process HTTP/HTTPS requests
  - Obtain parameters
  - Return results to the HTTP client (usually a web browser)
  - Maintain information across requests (Sessions)
- **Example:** `javax.servlet.http.HttpServlet`
  - `doGet(request, response)`
  - `doPost(request, response)`
- HTTP Methods

Servlets

- Java classes that run in a Web server
- Provide an API for HTTP interaction
  - Can process HTTP/HTTPS requests
  - Obtain parameters
  - Return results to the HTTP client (usually a web browser)
  - Maintain information across requests (Sessions)
- **Example:** `javax.servlet.http.HttpServlet`
  - `doGet(request, response)`
  - `doPost(request, response)`
- HTTP Methods
  - **GET**: pass parameters in URL

Servlets

- Java classes that run in a Web server
- Provide an API for HTTP interaction
  - Can process HTTP/HTTPS requests
  - Obtain parameters
  - Return results to the HTTP client (usually a web browser)
  - Maintain information across requests (Sessions)
- **Example:** `javax.servlet.http.HttpServlet`
  - `doGet(request, response)`
  - `doPost(request, response)`
- HTTP Methods
  - **GET**: pass parameters in URL
  - **POST**: pass parameters in the body of HTTP request

Servlet (2)

Servlet (3): Example

```
String persontype = request.getParameter("persontype");
String number = request.getParameter("name");
if (persontype.equals("student")) {
  // ... code to find students with the specified name ...
  // ... using JDBC to communicate with the database ...
  out.println("<table BORDER COLS=3>");
  out.println("<tr><td>ID</td><td>Name</td><td>Department</td></tr>");
  for (/* each result */) {
    // ... retrieve ID, name and dept name
    // ... into variables ID, name and deptname
    out.println("<tr><td>" + ID + "</td><td>" + name + "</td><td>" + deptname
  }
  out.println("</table>");
} else {
  // ... as above, but for instructors ...
```

Servlet (4): Sessions

- Servlet API supports handling of sessions

Servlet (4): Sessions

- Servlet API supports handling of sessions
- Sets a cookie on first interaction with browser, and uses it to identify session on further interactions

Servlet (4): Sessions

- Servlet API supports handling of sessions
- Sets a cookie on first interaction with browser, and uses it to identify session on further interactions
- To check if session is already active:

Servlet (4): Sessions

- Servlet API supports handling of sessions
- Sets a cookie on first interaction with browser, and uses it to identify session on further interactions
- To check if session is already active:
  - `if (request.getSession(false) == true)` $\Rightarrow$ existing session, else redirect to authentication

Servlet (4): Sessions

- Servlet API supports handling of sessions
- Sets a cookie on first interaction with browser, and uses it to identify session on further interactions
- To check if session is already active:
  - `if (request.getSession(false) == true)` ⇒ existing session, else redirect to authentication
  - `request.getSession(true)`: creates new session

Servlet (4): Sessions

- Servlet API supports handling of sessions
- Sets a cookie on first interaction with browser, and uses it to identify session on further interactions
- To check if session is already active:
  - `if (request.getSession(false) == true)` ⇒ existing session, else redirect to authentication
  - `request.getSession(true)`: creates new session
- Store/retrieve attribute value pairs for a particular session

Servlet (4): Sessions

- Servlet API supports handling of sessions
- Sets a cookie on first interaction with browser, and uses it to identify session on further interactions
- To check if session is already active:
  - `if (request.getSession(false) == true)` ⇒ existing session, else redirect to authentication
  - `request.getSession(true)`: creates new session
- Store/retrieve attribute value pairs for a particular session
  - `session.setAttribute("userid", userid)`

Servlet (4): Sessions

- Servlet API supports handling of sessions
- Sets a cookie on first interaction with browser, and uses it to identify session on further interactions
- To check if session is already active:
  - `if (request.getSession(false) == true)` ⇒ existing session, else redirect to authentication
  - `request.getSession(true)`: creates new session
- Store/retrieve attribute value pairs for a particular session
  - `session.setAttribute("userid", userid)`
  - `session.getAttribute("userid")`

Servlet (5): Support

- Servlets run inside application servers such as

Servlet (5): Support

- Servlets run inside application servers such as
  - Apache Tomcat, Glassfish, JBoss

Servlet (5): Support

- Servlets run inside application servers such as
  - Apache Tomcat, Glassfish, JBoss
  - BEA Weblogic, IBM WebSphere and Oracle Application Servers

Servlet (5): Support

- Servlets run inside application servers such as
  - Apache Tomcat, Glassfish, JBoss
  - BEA Weblogic, IBM WebSphere and Oracle Application Servers
- Application servers support

Servlet (5): Support

- Servlets run inside application servers such as
  - Apache Tomcat, Glassfish, JBoss
  - BEA Weblogic, IBM WebSphere and Oracle Application Servers
- Application servers support
  - deployment and monitoring of servlets

Servlet (5): Support

- Servlets run inside application servers such as
  - Apache Tomcat, Glassfish, JBoss
  - BEA Weblogic, IBM WebSphere and Oracle Application Servers
- Application servers support
  - deployment and monitoring of servlets
  - Java 2 Enterprise Edition (J2EE) platform supporting objects, parallel processing across multiple application servers, etc

Java Server Pages (JSP)

- A JSP page with embedded Java code

```
<html>
<head><title>Hello</title></head>
<body>
<% if (request.getParameter("name") == null) {
     out.println("Hello World");
   } else {
     out.println("Hello, " + request.getParameter("name"));
   } %>
</body>
</html>
```

- JSP is compiled into Java + Servlets
- JSP allows new tags to be defined, in tag libraries
  - such tags are like library functions, can are used for example to build rich user interfaces such

PHP

- PHP is widely used for Web server scripting
- Extensive libraries including for database access using ODBC

```
<html>
<head><title>Hello</title></head>
<body>
<?php
  if (!isset($_REQUEST['name'])) {
    echo "Hello World";
  } else {
    echo "Hello, " . $_REQUEST['name'];
  }
?>
</body>
</html>
```

USP of JSP

- **JSP vs. Active Server Pages (ASP)**

USP of JSP

- **JSP vs. Active Server Pages (ASP)**
  - ASP is a similar technology from Microsoft and is proprietary (uses VB)

USP of JSP

- **JSP vs. Active Server Pages (ASP)**
  - ASP is a similar technology from Microsoft and is proprietary (uses VB)
  - JSP is platform independent and portable

USP of JSP

- **JSP vs. Active Server Pages (ASP)**
  - ASP is a similar technology from Microsoft and is proprietary (uses VB)
  - JSP is platform independent and portable
- **JSP vs. Pure Servlets**

USP of JSP

- **JSP vs. Active Server Pages (ASP)**
  - ASP is a similar technology from Microsoft and is proprietary (uses VB)
  - JSP is platform independent and portable
- **JSP vs. Pure Servlets**
  - JSP is more convenient to write and modify regular HTML than using millions of println statements

USP of JSP

- **JSP vs. Active Server Pages (ASP)**
  - ASP is a similar technology from Microsoft and is proprietary (uses VB)
  - JSP is platform independent and portable
- **JSP vs. Pure Servlets**
  - JSP is more convenient to write and modify regular HTML than using millions of println statements
  - Web page designers can build the HTML, leaving places for servlet programmers to insert dynamic content

USP of JSP

- **JSP vs. Active Server Pages (ASP)**
  - ASP is a similar technology from Microsoft and is proprietary (uses VB)
  - JSP is platform independent and portable
- **JSP vs. Pure Servlets**
  - JSP is more convenient to write and modify regular HTML than using millions of println statements
  - Web page designers can build the HTML, leaving places for servlet programmers to insert dynamic content
- **JSP vs. JavaScript**

USP of JSP

- **JSP vs. Active Server Pages (ASP)**
  - ASP is a similar technology from Microsoft and is proprietary (uses VB)
  - JSP is platform independent and portable
- **JSP vs. Pure Servlets**
  - JSP is more convenient to write and modify regular HTML than using millions of println statements
  - Web page designers can build the HTML, leaving places for servlet programmers to insert dynamic content
- **JSP vs. JavaScript**
  - JavaScript can generate HTML dynamically on the client (Client Side)

USP of JSP

- **JSP vs. Active Server Pages (ASP)**
  - ASP is a similar technology from Microsoft and is proprietary (uses VB)
  - JSP is platform independent and portable
- **JSP vs. Pure Servlets**
  - JSP is more convenient to write and modify regular HTML than using millions of println statements
  - Web page designers can build the HTML, leaving places for servlet programmers to insert dynamic content
- **JSP vs. JavaScript**
  - JavaScript can generate HTML dynamically on the client (Client Side)
  - JSP are executed by the web server before HTTP response (Server Side) – can access server-side resources like databases

USP of JSP

- **JSP vs. Active Server Pages (ASP)**
  - ASP is a similar technology from Microsoft and is proprietary (uses VB)
  - JSP is platform independent and portable
- **JSP vs. Pure Servlets**
  - JSP is more convenient to write and modify regular HTML than using millions of println statements
  - Web page designers can build the HTML, leaving places for servlet programmers to insert dynamic content
- **JSP vs. JavaScript**
  - JavaScript can generate HTML dynamically on the client (Client Side)
  - JSP are executed by the web server before HTTP response (Server Side) – can access server-side resources like databases
- **JSP vs. Static HTML**

USP of JSP

- **JSP vs. Active Server Pages (ASP)**
  - ASP is a similar technology from Microsoft and is proprietary (uses VB)
  - JSP is platform independent and portable
- **JSP vs. Pure Servlets**
  - JSP is more convenient to write and modify regular HTML than using millions of println statements
  - Web page designers can build the HTML, leaving places for servlet programmers to insert dynamic content
- **JSP vs. JavaScript**
  - JavaScript can generate HTML dynamically on the client (Client Side)
  - JSP are executed by the web server before HTTP response (Server Side) – can access server-side resources like databases
- **JSP vs. Static HTML**
  - Regular HTML cannot contain dynamic information

Module Summary

Summary

- Familiarized with the Fundamentals notions and technologies of Web

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- Familiarized with the Fundamentals notions and technologies of Web
- Learnt about Scripting

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

Summary

- Familiarized with the Fundamentals notions and technologies of Web
- Learnt about Scripting
- Learnt the notions of Servlets

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.